

Permission-Aware Retrieval for Multi-Tenant Agentic AI Systems

A practical architecture for secure RAG, RBAC enforcement, and zero-leakage retrieval in enterprise SaaS

Date: February 6, 2025

Author: Toyin Bakare

Abstract

Agentic AI systems operating in multi-tenant SaaS environments introduce new security risks—particularly when retrieval steps occur before permissions are enforced. This white paper presents a permission-first retrieval architecture designed to eliminate cross-tenant and intra-tenant leakage while maintaining high recall and strong performance. We define common failure modes, propose a reference design, and outline operational governance practices for real-world deployment.

1. Introduction

Modern AI assistants and retrieval-augmented generation (RAG) pipelines routinely access heterogeneous data sources, including customer records, internal notes, and knowledge bases. Traditional search systems assume that the caller has full access to the index—an assumption that breaks down entirely in multi-tenant SaaS. Without architectural safeguards, retrieval can bypass tenant boundaries or surface unauthorized content.

This paper outlines how to design retrieval layers that enforce permissions *before* any vector search or ranking step occurs.

2. Problem Statement

Multi-tenant SaaS systems serve multiple customers on shared infrastructure, each with object-level, row-level, and field-level entitlements. Agentic AI amplifies the security risks, issuing numerous retrieval calls per user request and potentially across domains with differing security models.

A secure solution must satisfy two principles:

1. **High recall** for relevant, allowed data
2. **Provable permission enforcement** at the earliest possible stage

Post-filtering—removing restricted items only after retrieval—fails both requirements due to leakage through logs, ranking metadata, or intermediate model prompts.

3. Threat Model and Failure Modes

Common failure patterns include:

- **Cross-tenant leakage** due to missing `tenant_id` constraints
- **Intra-tenant leakage** via unauthorized access to restricted cases or notes
- **Side-channel leakage**, such as ranking signals or snippet text revealing sensitive details
- **Agent misuse**, where tools invoke retrieval with elevated service credentials

A key insight: *Permissions applied after vector retrieval are insufficient and unsafe*. Even filtered-out results may have already influenced an embedding model or appeared in telemetry.

4. Reference Architecture: Permission-First Retrieval

A robust design must transform user identity into a permission-scoped retrieval envelope *before* candidate generation.

4.1 Identity and Session Context

- `tenant_id`
- `user_id`
- `role_ids`
- `group_ids`
- Session claims and delegated authority

4.2 Policy Evaluation Layer

- Object-level ACLs
- Row-level entitlements
- Sharing rules

- Conditional access policies

4.3 Pre-Filtering

Apply metadata constraints before vector scoring:

- Tenant boundaries
- Record-owner constraints
- Visibility rules
- Data-classification flags

4.4 Retrieval and Ranking

Only after permitted candidates are computed should vector similarity or hybrid ranking run.

4.5 Auditability

Maintain immutable logs capturing:

- Input query
- Identity context
- Policy decisions
- Candidate set size
- Allowed/denied counts

5. Operational Practices and Governance

Automated Leakage Tests

Regular canary queries designed to fail securely (e.g., cross-tenant attempts).

Security Review Gates

Every new data source or embedding index undergoes architecture review.

Observability

Track:

- Denied retrieval counts

- Per-tenant error rates
- Index drift or schema changes

Red Teaming

Adversarial prompt tests to force agent escalation or boundary bypass attempts.

6. Conclusion

Permission-aware retrieval is mandatory for secure enterprise AI adoption. A permission-first architecture eliminates leakage pathways, builds customer trust, and ensures that AI-powered workflows comply with stringent enterprise access-control models.

References

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. [Foundational theory for MRR and Top-K metrics].
- [2] A. Burkov, *Machine Learning Engineering*. True Positive Inc., 2020. [Reference for model versioning and deployment patterns].
- [3] N. Thakur, N. Reimers, A. Daxenberger, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Information Retrieval," in *Proc. 35th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021. [The industry standard for "Golden Set" construction in embedding evaluation].
- [4] J. Lin, R. Nogueira, and A. Yates, *Pretrained Transformers for Text Ranking: BERT and Beyond*. Morgan & Claypool Publishers, 2021. [Technical depth on semantic shift and ranking regressions].
- [5] K. W. Moe, *Vector Databases: Architecture and Applications*. Sebastopol, CA: O'Reilly Media, 2025. [Guidelines for Shadow Indices and vector namespace isolation].
- [6] L. Chen et al., "MLOps: Continuous Delivery and Automation Pipelines in Machine Learning," *IEEE Software*, vol. 39, no. 2, pp. 42-50, 2022. [Protocol for promotion and rollback playbooks].